



Руководство по скриптам автоматизации

i8080-5 Master Controller v1.3

Содержание

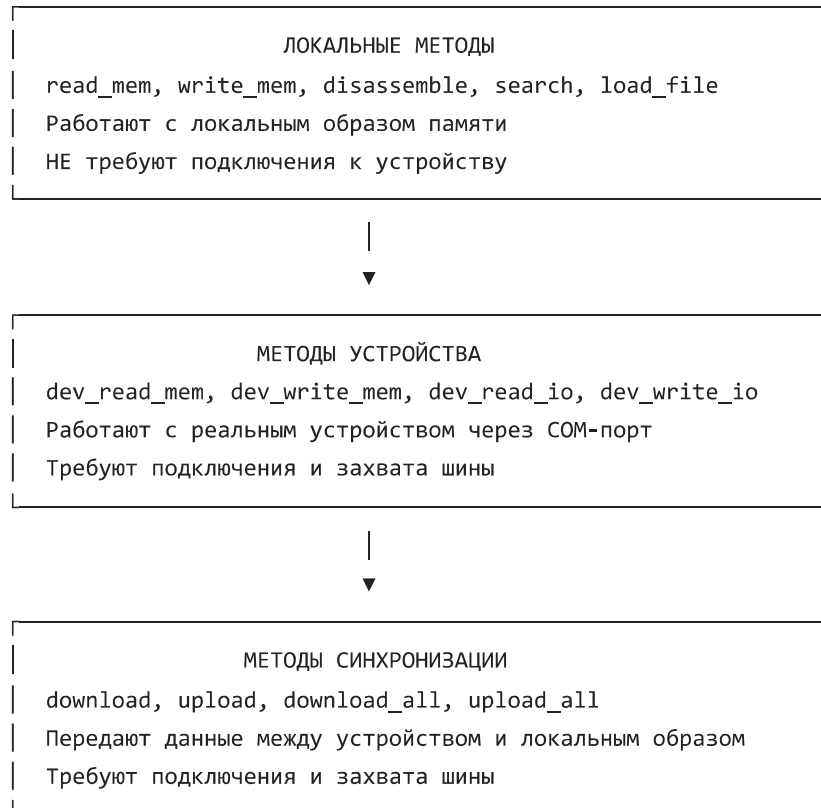
1. Введение
2. Основы работы
3. Справочник API
4. Примеры скриптов
5. Советы и лучшие практики

1. Введение

Скрипты автоматизации позволяют управлять программой i8080-5 Master Controller с помощью Python-кода. Вы можете:

- **Автоматизировать рутинные операции** — заполнение памяти паттернами, поиск, патчинг
- **Тестировать устройства** — запись и чтение памяти, тестирование IO-портов
- **Создавать сложные сценарии** — считывание прошивки, модификация, загрузка обратно
- **Анализировать код** — дизассемблирование, поиск инструкций, подсчёт команд

Архитектура API



2. Основы работы

Перейдите на вкладку "**Scripts**" (или "**Скрипты**" в русской локализации).

1. Введите код в редактор
2. Нажмите "► **Run Script**"
3. Результаты появятся в области **Output**

Используйте `print()` для вывода информации:

```
print("Hello from script!")
print(f"Value: {0x55:02X}")
```

3. Справочник API

Управление шиной

Метод	Описание	Возвращает
<code>hold_bus()</code>	Отправить команду захвата шины	True если отправлена
<code>unhold_bus()</code>	Отправить команду освобождения шины	True если отправлена
<code>wait_bus(timeout=5.0)</code>	Ждать захвата шины	True если захвачена
<code>wait_unhold(timeout=5.0)</code>	Ждать освобождения шины	True если освобождена

```
hold_bus()
if wait_bus():
    print("Bus is active!")
    # ... работаем с устройством ...
    unhold_bus()
    wait_unhold()
```

Локальная память (не требует подключения)

Метод	Описание
<code>read_mem(addr)</code>	Прочитать байт из локального образа
<code>write_mem(addr, val)</code>	Записать байт в локальный образ
<code>read_block(addr, size)</code>	Прочитать блок из локального образа
<code>write_block(addr, data)</code>	Записать блок в локальный образ
<code>fill_mem(addr, size, val)</code>	Заполнить диапазон значением

Память устройства (требует шину)

Метод	Описание
<code>dev_read_mem(addr)</code>	Прочитать байт из памяти устройства
<code>dev_write_mem(addr, val)</code>	Записать байт в память устройства
<code>dev_read_io(port)</code>	Прочитать из IO-порта устройства
<code>dev_write_io(port, val)</code>	Записать в IO-порт устройства

Синхронизация (устройство ↔ локальный образ)

Метод	Описание
<code>download(addr, size)</code>	Считать блок из устройства в локальный образ
<code>upload(addr, size)</code>	Записать блок из локального образа в устройство

<code>download_all(start, end)</code>	Считать всю память устройства
<code>upload_all()</code>	Записать весь локальный образ в устройство

Дизассемблирование и поиск

Метод	Описание
<code>disassemble(addr, length, show=False)</code>	Дизассемблировать локальный образ. <code>show=True</code> обновляет окно дизассемблера
<code>search(pattern, mode)</code>	Поиск. <code>mode</code> : "hex" или "ascii"

Файлы и утилиты

Метод	Описание
<code>load_file(path, base_addr=0)</code>	Загрузить файл в локальный образ
<code>save_file(path)</code>	Сохранить локальный образ в файл
<code>status()</code>	Состояние программы (словарь)
<code>log(msg)</code>	Вывод в журнал программы
<code>goto(addr)</code>	Переход к адресу в hex-редакторе
<code>refresh()</code>	Принудительное обновление GUI

4. Примеры скриптов

Пример 1: Создание тестовой программы

```
program = [  
    0x3E, 0x55,      # MVI A, 55h  
    0xD3, 0x01,      # OUT 01h  
    0x3E, 0xAA,      # MVI A, AAh  
    0xD3, 0x01,      # OUT 01h  
    0xC3, 0x00, 0x00 # JMP 0000h  
]  
  
write_block(0x0000, program)  
print("Test program written to 0x0000")  
  
lines = disassemble(0x0000, len(program), show=True)  
for line in lines:  
    print(line)
```

Пример 2: Поиск и анализ инструкций

```
jmps = search('C3', 'hex')  
print(f"Found {len(jmps)} JMP instructions:")  
  
for addr in jmps:  
    low = read_mem(addr + 1)  
    high = read_mem(addr + 2)  
    target = low | (high << 8)  
    print(f" 0x{addr:04X}: JMP 0x{target:04X}")
```

Пример 3: Полный цикл работы с устройством

```

info = status()
if not info['connected']:
    print("ERROR: Device not connected!")
else:
    hold_bus()
    if wait_bus():
        data = download(0x0000, 256)
        print(f"Downloaded {len(data)} bytes")
        unhold_bus()
        wait_unhold()

    # Обработка локально
    nops = search('00', 'hex')
    for addr in nops[:10]:
        write_mem(addr, 0x3E)
        write_mem(addr + 1, 0x55)

    disassemble(0x0000, 256, show=True)

    hold_bus()
    if wait_bus():
        upload(0x0000, 256)
        unhold_bus()
        wait_unhold()

```

Пример 4: Тестирование IO-портов

```

hold_bus()
if wait_bus():
    for val in [0x00, 0xFF, 0x55, 0xAA]:
        dev_write_io(0x01, val)
        print(f"IO 0x01 = 0x{val:02X}")

```

```
result = dev_read_io(0x02)
if result is not None:
    print(f"IO 0x02 = 0x{result:02X}")

unhold_bus()
wait_unhold()
```

5. Советы и лучшие практики

ПЛОХО: работа с устройством без проверки шины

```
dev_write_io(0x01, 0x55) # Может вызвать ошибку!
```

ХОРОШО: проверка подключения и шины

```
if status()['connected']:
    hold_bus()
    if wait_bus():
        dev_write_io(0x01, 0x55)
    unhold_bus()
    wait_unhold()
```

ПЛОХО: постоянное обновление GUI тормозит


```
for i in range(1000):  
    write_mem(i, 0x00) # Каждый раз обновляет GUI
```

ХОРОШО: обновляем один раз в конце

```
for i in range(1000):  
    write_mem(i, 0x00)  
refresh() # Обновляем GUI один раз
```

Совет: используйте `disassemble()` для анализа (без GUI) и `show=True` для отображения результата в окне дизассемблера.

Горячие клавиши

Клавиша	Действие
Ctrl+O	Открыть файл прошивки
Ctrl+S	Сохранить дамп
Ctrl+F	Поиск по памяти
Ctrl+G	Переход к адресу
Ctrl+E	Экспорт дизассемблера

Ctrl+D	Дизассемблировать
Ctrl+H	Захват шины (HOLD)
Ctrl+U	Освобождение шины (UNHOLD)
Ctrl+Z / Ctrl+Y	Отмена / Повтор
F5	Обновить

Руководство по скриптам автоматизации для i8080-5 Master Controller v1.3